TrajectAI

Thématique MOVECITY « Une ville qui fluidifie les mobilités quotidiennes »

Livrable J4 : Jalon 1

Conception UX/UI, architecture et sécurité

Wireframes · Architecture technique v1 · Cartographie des risques (threat model v1) · Identité graphique v1

Élément	Détail
Projet	TrajectAI — optimisation explicable des trajets multi-étapes
Équipe	Kavisha RUBAKANTHAN Nono Yvana NOUTCHANG Luân TA · Ywen MASSON Noussirath BELLO Cyril DABERNARD
Date	6 octobre 2026
Plateforme	https://trajectai-paris.vercel.app
Tableau de bord	Trello — https://trello.com/b/VhnwiHpv/tableau-de-bord-dequipe

Introduction

TrajectAI répond à une problématique simple : **comment l'IA peut-elle optimiser anonymement les trajets multi-étapes des actifs et des familles, tout en leur laissant le contrôle total sur leurs données et leurs choix de déplacement ?**

Notre principe directeur guide chaque décision de conception présentée dans ce document : **le citoyen liste, l'IA optimise, la ville informe, l'usager contrôle.**

Ce livrable formalise la conception de la plateforme à l'issue de la journée J4. Chaque filière y présente sa production, et toutes les décisions ont été prises de façon concertée, afin qu'aucun choix structurant ne reste isolé dans une seule filière.

Partie	Contenu	Filière responsable
1. Wireframes	Parcours utilisateur et structure des écrans clés	Développement web
2. Architecture technique v1	Composants, flux, choix de données et de technologies	Développement web
3. Cartographie des risques	Actifs, flux, points d'entrée et threat model v1	Cybersécurité
4. Identité graphique v1	Logo, couleurs, typographie, accessibilité, règles de datavisualisation	Marketing digital

1. Wireframes

Les wireframes représentent la structure des écrans, sans couleurs ni décoration, pour valider l'ergonomie avant le design. Nous les avons conçus **en version mobile d'abord** : nos utilisateurs, des actifs et des parents de 20 à 45 ans, organisent leur journée sur leur téléphone, souvent dans l'urgence.

1.1 Parcours utilisateur

Le parcours suit exactement notre principe directeur, en quatre temps :

1. **Lister** : l'usager saisit ses démarches dans n'importe quel ordre, ajoute une contrainte horaire éventuelle (« école avant 8h30 »), puis choisit son mode de transport et sa priorité.
2. **Optimiser** : en un clic, l'IA teste tous les ordres possibles et retient le meilleur.
3. **Informer** : l'application affiche l'ordre recommandé, l'heure de chaque étape et une explication en une phrase pour chaque choix.

4. **Contrôler** : l'usager change sa priorité ou déplace une étape, et le calcul se met à jour instantanément. Il peut effacer son trajet à tout moment.

1.2 Écrans clés

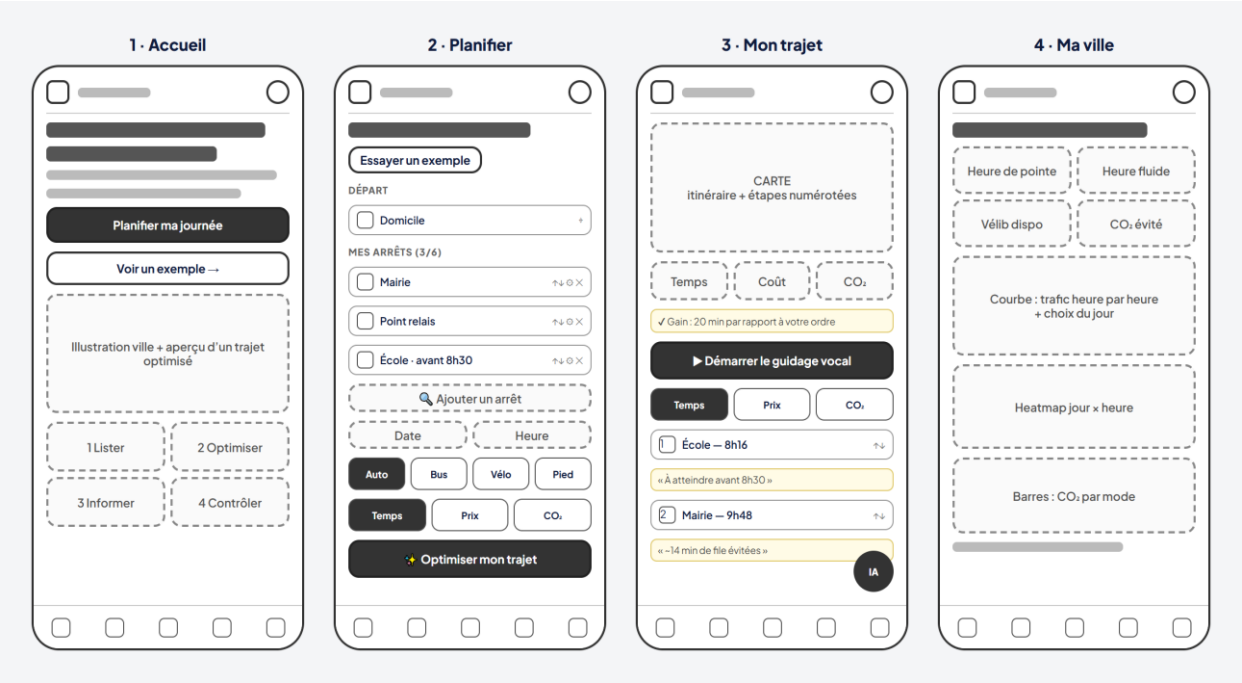


Figure 1 Wireframes des quatre écrans principaux (version mobile)

Écran	Objectif	Éléments clés
Accueil	Comprendre la promesse en 5 secondes et passer à l'action	Accroche, bouton « Planifier ma journée », exemple en un clic, les 4 étapes du principe, engagements de confiance
Planifier	Saisir sa journée sans effort	Départ (ou « ma position »), arrêts avec type de lieu détecté automatiquement, contraintes horaires, date et heure, mode, priorité
Mon trajet	Comprendre et ajuster la proposition de l'IA	Carte, temps, coût et CO_2, gain par rapport à l'ordre saisi, explication par étape, réorganisation, comparaison des modes, guidage vocal
Ma ville	Rendre lisibles les données de mobilité	Indicateurs clés, trafic heure par heure, carte de chaleur jour × heure, CO_2 par mode, Vélib' en temps réel

1.3 Principes d'ergonomie retenus

- **Trois actions suffisent** pour obtenir un trajet optimisé : charger ou saisir ses étapes, choisir sa priorité, cliquer sur « Optimiser ».
- **Navigation au pouce** : barre d'onglets fixée en bas de l'écran sur mobile, menu horizontal sur ordinateur.

- **L'explication est au plus près du choix** : chaque étape porte sa justification, sans écran supplémentaire.
- **Assistant vocal accessible partout** grâce à un bouton flottant, pour les parents qui ont les mains prises.
- **Aucune donnée demandée pour essayer** : le compte est facultatif.

2. Architecture technique v1

L'architecture repose sur un principe de **protection des données dès la conception** (« privacy by design ») : le serveur qui calcule les trajets ne possède aucune base de données. Les trajets ne peuvent donc pas être conservés, même par erreur.

2.1 Vue d'ensemble

1. Le navigateur (React + Tailwind)

L'utilisateur arrive sur une interface responsive pensée mobile d'abord, avec cinq pages : Accueil, Planifier, Mon trajet, Ma ville, Compte. Le trajet calculé reste en mémoire le temps de la session et disparaît à la fermeture de l'onglet aucune persistance côté client. La carte s'appuie sur Leaflet et OpenStreetMap. Pour la voix et la géolocalisation, tout est traité localement dans le navigateur (Web Speech API, géolocalisation à la demande)

2. L'API TrajectAI (Node.js/Express, déployée sur Vercel, région Paris)

cette API **n'a pas de base de données**, ne journalise aucune requête, valide les entrées avec zod et applique du rate limiting et helmet. C'est un choix d'architecture qui élimine une bonne partie des risques classiques (fuite de logs, rétention de données personnelles) avant même d'avoir à les corriger.

3. Supabase (hébergé en UE)

Rôle volontairement restreint : authentification par email/mot de passe (hash bcrypt) et une seule table, preferences, protégée par Row Level Security chaque utilisateur ne peut lire ou écrire que ses propres préférences. Aucune donnée de trajet n'y transite.

2.2 Composants

Composant	Technologie	Rôle et justification
Interface web	React 19, Vite, Tailwind CSS	Site responsive, mobile d'abord. Technologies très répandues, rapides et bien documentées.
Carte	Leaflet + OpenStreetMap	Affichage de l'itinéraire. Solution gratuite et libre, sans clé payante.
Graphiques	Recharts	Visualisations interactives du dashboard, avec infobulles.

Composant	Technologie	Rôle et justification
API	Node.js + Express	Même langage que l'interface. Sans état et sans base de données.
Moteur d'optimisation	Algorithme développé par l'équipe	Teste tous les ordres possibles (jusqu'à 720 pour 6 étapes) et justifie chaque choix par des chiffres.
Assistant IA	GPT-OSS 120B (open source) via Groq	Répond aux questions à l'écrit ou à la voix. Un moteur de règles local prend le relais en cas de panne.
Comptes	Supabase (hébergement UE)	Authentification sécurisée et stockage des seules préférences.
Hébergement	Vercel, région Paris	Déploiement continu, interface et API sous la même adresse.

2.3 Choix des données

Conformément au cahier des charges, nos visualisations et calculs s'appuient sur des **données ouvertes réelles**, mises au service d'une expérience utile et pas seulement d'un affichage.

Donnée	Source	Usage dans TrajectAI
Débit et taux d'occupation des routes, heure par heure	Ville de Paris Comptages routiers permanents (ODbL), près de 30 millions de mesures	Ajuste les temps de trajet selon l'heure ; dashboard
Disponibilité Vélib'	Vélib' Métropole temps réel (ODbL)	Dashboard
Émissions de CO_2 par mode	ADEME Impact CO_2	Calcul de l'empreinte de chaque option ; dashboard
Temps et distances de trajet	OSRM / OpenStreetMap	Matrice des trajets entre les étapes
Adresses et lieux	Photon (OpenStreetMap) + Base Adresse Nationale	Recherche et détection du type de lieu
Affluence des guichets et commerces	Profils types modélisés par l'équipe	Estimation de la file d'attente selon l'heure

2.4 Organisation du code

Le code est versionné dans un seul dépôt, séparé en trois parties :

```
TrajectAI/
├── client/     Interface React : pages, composants (carte, assistant…), appels à l'API
├── server/     API Node.js / Express : routes, moteur d'optimisation, accès aux données
├── supabase/   Script de création de la table preferences et de ses règles de sécurité
└── api/        Point d'entrée pour le déploiement sur Vercel
```

2.5 Routes de l'API

Route	Rôle
POST /api/optimize	Reçoit les étapes, le départ, le mode et la priorité ; renvoie l'ordre optimal expliqué
GET /api/geocode	Recherche un lieu ou une adresse et détecte son type (mairie, école…)
GET /api/city/…	Fournit les données ouvertes du dashboard (trafic, Vélib', CO_2)
POST /api/assistant	Fait répondre l'assistant IA aux questions de l'usager

2.6 Déroulé d'une optimisation

1. L'usager envoie ses étapes, son heure de départ, son mode et sa priorité.
2. L'API vérifie la requête, puis calcule les temps et distances entre tous les points.
3. Pour chaque ordre possible, elle simule la journée : trafic à l'heure concernée, horaires d'ouverture, file d'attente, contraintes.
4. Le meilleur ordre selon la priorité est retenu, avec une explication pour chaque étape.
5. Le résultat est renvoyé au navigateur. Rien n'est enregistré sur le serveur.

3. Prototype squelette

Le prototype squelette est en ligne : **https://trajectai-paris.vercel.app**. Il valide l'architecture de bout en bout, de l'interface jusqu'aux données ouvertes.

Élément	État
Navigation et pages principales (Accueil, Planifier, Mon trajet, Ma ville, Compte)	En place
Parcours complet : saisir les étapes → optimiser → afficher le trajet expliqué	Fonctionnel
Connexion à l'API et aux données ouvertes (trafic, Vélib', CO_2)	Fonctionnel
Comptes utilisateurs (Supabase)	Fonctionnel
Affichage sur mobile (responsive)	Vérifié
Déploiement en ligne (Vercel)	En place

Prochaines étapes (Partie 2)

- Stabiliser les fonctionnalités prioritaires après la revue du prototype.
- Intégrer l'authentification sécurisée et les corrections issues de l'audit de sécurité.